StateGuard: Failure Propagation and State-Consistent Recovery in Tool-Using Multi-Agent Workflows

Xinying Liu Jingwen Zhang
University of California San Diego, CSE 291A — Option 5
xil277@ucsd.edu jiz333@ucsd.edu

Abstract. StateGuardstudies how multi-agent failures propagate into persistent state and how recovery can preserve legitimate intervening changes. We implement a deployment workflow with versioned state, provenance, and fault injection, then record 99 real-model clean/fault pairs on AWS. Across 30 combinations valid for every policy, task completion is 30% without recovery, 100% with full restart, and 80% with intermediate verification. Restart approximately doubles tokens and latency. Most injections do not propagate, limiting recovery claims. The tested selective-recovery implementation provides the basis for Phase 2 evaluation.

1 Problem and project scope

In a deployment workflow, a Config Agent may report the wrong database port, a Planner may incorporate it, and an Executor may commit the resulting configuration. Recomputing the plan leaves the committed error behind; restoring an old snapshot may erase a later administrator update. We ask whether dependency tracking and current-state validation can identify reusable work and repair the rest safely and efficiently.

Our environment contains API, worker, and database services with checkable goals and protected authentication/audit fields. Config, Dependency, and Resource Agents produce structured results; a Planner consumes their outputs; a controlled Executor applies writes. Versioned deployment knowledge supplies stale-source cases. Python orchestrates the workflow, an independent adapter validates model responses and logs prompts, responses, usage, and bounded retries, and SQLite stores versions and action records. A deterministic backend tests system semantics independently of model behavior.

2 State and recovery design

For each field k, the store maintains $S(k) = (x_k, v_k)$. A result records its identifier, parents, read set $R_r = \{(k, v)\}$, status, and cost. A typed provenance graph connects results, actions, state versions, and external events through consumption, reads, triggers, and committed writes. This links a corrupted result to downstream state events. The Planner records all three analysis inputs as parents; provenance is conservative at result granularity and does not infer exact semantic use of every field.

Given a faulty result f, the engine invalidates $I(f) = \{f\} \cup \mathrm{Desc}(f)$. A result is a reuse candidate only if

$$r \notin I(f) \ \wedge \ \forall (k, v) \in R_r : \ v_k^{\text{current}} = v. \tag{1}$$

This is an admission condition, not a proof of semantic correctness. Missing provenance, affected actions, and later writes require further checks. Invalid analyses and the Planner are recomputed; valid independent analyses may be retained.

Tool writes carry an expected version and action ID. A SQLite transaction checks the version and commits state, ledger, and event records together. An identical retry returns its recorded outcome, including after a simulated lost acknowledgement. This guarantee covers controlled local tools, not arbitrary remote services. Selective replay writes only changed fields and blocks on conflicting updates. Irreversible effects remain visible and preclude claiming complete recovery.

Full restart runs a fresh attempt and replays recorded external events in order, checking their original value/version preconditions; it cannot discard administrator updates. Integration tests cover independent-result reuse, same-field conflicts, replay precondition failures, and deduplication. The integrated suite passes 156 tests. These tests supply a Phase 2 foundation; the measurements below concern Phase 1 baselines.

3 Experimental protocol

We compare no recovery (NR), full restart (FR), and always verify (AV). AV checks analysis outputs against simulated truth and permits one retry: its local verifier is an optimistic oracle, not a learned detector. FR receives a clean recovery attempt after the faulty execution. Recovery uses the harness's fault information; detection quality is outside this evaluation.

The matched matrix uses one task, 11 fault cases, three seed labels $(7, 11, 19)$, and three policies: $11 \times 3 \times 3 = 99$ pairs. Each pair has a clean control and an injected run; FR also records its restart. Cases include incorrect ports, stale protocol results, analysis timeouts, malicious protected-field instructions, and two intended ignored controls. The model is TritonAI `api-glm-5.3-flash`; orchestration runs on an AWS CPU instance. Every policy allows at most three API attempts per request with a five-second minimum request interval. Seeds fix the fault schedule, not provider sampling. Earlier smoke and paired pilots remain available, but their different retry settings are not pooled into this matrix.

4 Results and interpretation

All 99 pairs were recorded; 96 are individually valid. Two FR pairs in seed 7 contain provider HTTP 422 errors. One NR pair in seed 19 has a failed clean control without an API error. We retain these traces and exclude their case/seed combinations from every policy's comparison, leaving $9+11+10 = 30$ matched combinations per policy. These exclusions yield a conditional comparison, not evidence that clean execution is always reliable.

Table 1: Matched outcomes and fault-side cost ($n = 30$ per policy). Cost includes recovery, but excludes the separately logged clean control.

Metric	NR	FR	AV
Completed tasks	9/30	30/30	24/30
Unauthorized commits	0	0	0
Mean tokens	756.7	1553.8	859.8
Mean latency (s)	19.46	39.04	24.19
Mean model calls	3.8	7.8	4.8
Mean recomputed stages	0	4	0

FR completes all matched tasks at 2.05× NR's token use and 2.01× its latency. AV completes 24 tasks at 1.14× the tokens and 1.24× the latency. Request pacing is included in latency; these values describe the configured end-to-end system, not model compute time alone. This is a descriptive pilot rather than a statistical generalization over deployment tasks.

NR classifies 4 injections as consumed, 9 ignored, 11 refused, and 6 blocked. These labels describe propagation, not final recovery success. All six adversarial runs succeed under every policy because the instructions are ignored. Zero unauthorized commits therefore does not demonstrate recovery from an accepted attack. Separate stochastic model calls can also change propagation labels across policies.

The persistent-port case gives the clearest propagation evidence: NR consumes the fault and fails in all three runs; FR completes all three; AV remains blocked in all three. AV also fails all three compatibility-flag controls, which the model refuses rather than ignores. Among accidental cases, completion is 3/24 for NR, 24/24 for FR, and 18/24 for AV. The aggregate AV score thus conceals systematic failure on two case families.

5 Limits and next experiments

Phase 1 delivers the workflow, model adapter, fault harness, checkers, typed traces, three executed baselines, and auditable artifacts. Real-model evidence covers one task and one dependency topology. Schema checks and simulated tools constrain behavior; AV sees ground truth; automatic detection and separately isolated tracking overhead are not evaluated. SQLite tests establish specific recovery semantics, not arbitrary rollback of a browser or remote API.

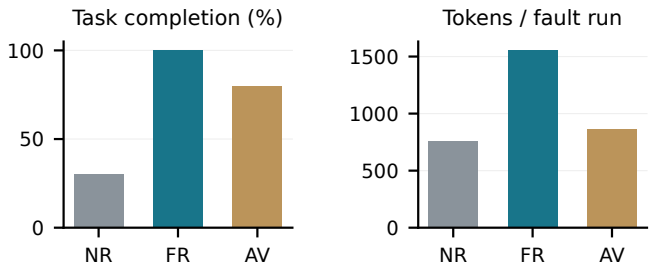


Figure 1: Completion and token cost on the same 30 valid case/seed combinations. These heterogeneous cases are not independent samples from a population of deployment tasks.

Phase 2 will compare dependency-only replay and state-aware selective recovery against the existing baselines under identical fault identities, external-event schedules, and budgets. We will vary chain, fan-out, and diamond dependencies; unrelated and same-field administrator updates; and shared-resource conflicts. The target is correctness with preserved legitimate updates, followed by reduced recomputation. Ablations remove read-version validation and action deduplication. We will measure stale reuse, retained valid work, recovery latency, actual tokens, tracking overhead, and residual irreversible effects. A corrected configuration will not erase evidence of an unrecoverable action.

Appendix: timeline and milestones. Nov. 3–4: freeze Phase 1 code and evidence. **Nov. 5–15:** extend topology and external-event workloads; validate replay oracles. **Nov. 16–25:** run selective-recovery comparisons and ablations, preserving failed/blocked cases. **Nov. 26–Dec. 4:** analyze failure modes and overhead; add a controlled-service case if core results are stable. **Dec. 5–10:** reproduce results and complete the final report.

Related work and artifacts. AgentDojo evaluates utility and injection in tool-using agents [1]; TAMAS supplies adversarial multi-agent scenarios [2]; A2ASecBench studies protocol attacks with joint safety–utility evaluation [3]. We focus on known-fault recovery under changing state. The artifact repository contains source, manifests, paired traces, CSVs, and figures; the matched experiment uses revision `6930ac8`.

References

[1] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv:2406.13352*, 2024.

[2] Microsoft. TAMAS. https://github.com/microsoft/TAMAS.

[3] T. Li et al. A2ASecBench: A protocol-aware security benchmark for agent-to-agent multi-agent systems. *ICLR*, 2026.